

1 Why This Course Exists

In this session:

1.1	What Is Machine Learning?	1
1.2	The Cheap Habit	3
1.3	What the Notebook Showed	4
1.4	Our Approach	4
1.5	The Modern Engineer's Mandate	5
1.6	Prerequisites and Logistics	5
1.7	Exercises	6

Session 0

One-line motivation. You already know how to call `sklearn`. This session is about why calling `sklearn` is not the same as understanding machine learning—and about what we intend to teach you instead.

This session has a companion Jupyter notebook. Run it before reading these notes, or alongside them:

Session 00 — Live Notebook

In the notebook you will find:

- the projectile data generator and baseline features;
- the conventional ML zoo pipeline (Linear, Polynomial, Ridge, Random Forest);
- the silent out-of-distribution failure demo;
- the physically grounded trigonometric reconstruction.

The notebook contains a full demonstration of the failure mode these notes are about. We will not repeat the code here; we will explain what it means.

1.1 What Is Machine Learning?

Machine learning is the process of learning useful structure from data. We observe some inputs and outputs, and we want a program that maps new inputs to correct outputs.

Classical Software Engineering vs. Machine Learning

In classical software engineering, a human expert encodes domain knowledge into a deterministic algorithm. The engineer designs a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that maps inputs to outputs using explicit logic. The computer executes it. This works beautifully for well-defined problems: computing taxes, sorting a list, calculating a trajectory. It fails when the rules are unknown, too numerous, or too subtle to articulate.

Machine learning *inverts* the process. Instead of writing the function f by hand, we:

1. Choose a family of candidate functions—a *hypothesis space* $\mathcal{F} = \{f_\theta \mid \theta \in \Theta\}$.

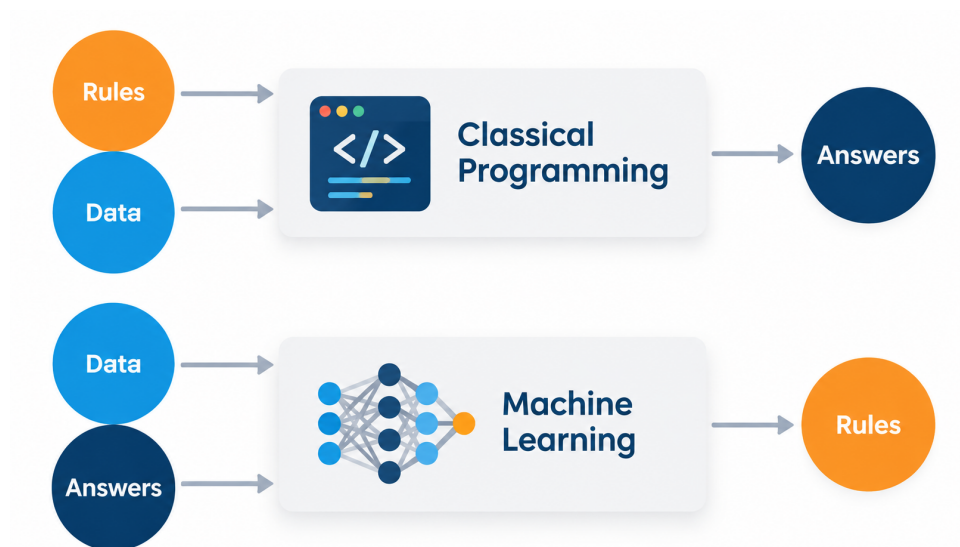


Figure 1: Comparison of the traditional programming paradigm (top) where rules are explicit, versus the machine learning paradigm (bottom) where rules are inferred from data.

2. Collect examples—a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ of input–output pairs.
3. Define a loss function $\mathcal{L}$ that measures how wrong a candidate is.
4. Search for the parameters θ^* that make the function as correct as possible.

In compact form:

$$\text{ML: } (\mathcal{D}, \mathcal{F}, \mathcal{L}) \xrightarrow{\text{Optimizer}} \theta^* \implies f_{\theta^*}(x) \approx y \quad (1)$$

Later we will talk about different parts of this formula.

Analogy. Classical programming is like giving someone a recipe. Machine learning is like giving them a kitchen, ingredients, and a way to measure whether the food tastes good—and letting them figure out the recipe themselves.

Newton Was Doing Machine Learning

Before machine learning had a name, scientists were already doing it.

Between 1609 and 1619, Kepler published three empirical laws of planetary motion. The third is the cleanest: the square of a planet’s orbital period is proportional to the cube of its semi-major axis:

$$T^2 \propto a^3.$$

Kepler did not know why. He had found a pattern in Tycho Brahe’s data. This is descriptive learning. It is what a regression model does.

Newton, in 1687, proposed a mechanism: an inverse-square force law,

$$F \propto \frac{1}{r^2}.$$

From this single law, plus his three laws of motion, he *derived* Kepler’s third law. It was not a

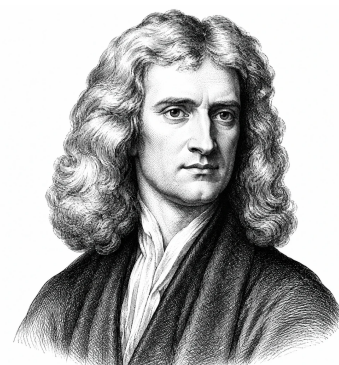


Figure 2: Isaac Newton (1643–1727), who restricted his hypothesis space by physical intuition rather than brute-force fitting.

fit. It was a consequence. And when he tested the same law against the Moon's orbit, the tides, and the precession of the equinoxes—phenomena he had not used to derive it—it worked.

Newton had three things a modern regressor does not.

First, a **tiny hypothesis space**. He considered algebraic force laws of the form $F \propto r^n$ for a small set of n . The whole space could be written on a blackboard.

Second, an **independent verification**. He did not test on the same data he used to derive the law. He tested on the Moon, on tides, on precession. These were chosen by reasoning about what a gravitational law *should* affect.

Third, a **mechanism**. The law was not just accurate. It was explanatory. It connected planets to falling apples in a single framework.

Machine learning is not a departure from classical science. It is the automation of hypothesis selection at scale. But automation is a tool, not a substitute for understanding. Newton automated nothing, but he understood everything he did. We can automate everything and understand nothing.

1.2 The Cheap Habit

The companion notebook contains a full demonstration of a projectile-range problem treated as a conventional machine learning task. The instructor tries linear regression, tree-based models, polynomial features, ridge regularization and finally a model zoo. Every model achieves acceptable scores. The 3rd degree polynomial achieves a test R^2 of 0.998. The demo looks like a success.

Then the notebook steps out of the role and asks what actually happened. The answer is: nothing was understood, and the model would fail catastrophically outside the range where it was tested.

Let us name the practice.

Definition 1.1: The Cheap Habit

The **cheap habit** is the practice of selecting, tuning, and deploying models by numerical score, without understanding what the model assumes, what structure the phenomenon has, or whether the two are compatible.

The cheap habit is not stupidity. It is a natural response to a real problem. Modern ML libraries make it extremely easy to run dozens of models in a single afternoon. The APIs are clean, the documentation is good, and the results often look acceptable. Under time pressure—and engineering is always under time pressure—the cheap habit is the path of least resistance.

It is also a trap. It fails in four predictable ways.

1. It cannot detect silent failures.

A model can have excellent test metrics and still be wrong. It can latch onto a spurious feature, fail to extrapolate, or collapse under a distribution shift that no held-out sample captured. Metrics cannot see any of these failures. Only understanding can.

2. It cannot be defended.

When a model fails in production, the review committee will ask why this model, why this hypothesis class, what it assumes, and why the assumption is justified. “It had the best score” is not an acceptable answer.

3. It cannot be improved.

If you do not know why a model works, you cannot make it work better. You can only try more models. This is called *hyperparameter tuning*, and it is the cheapest possible form of engineering: no insight, no principle, only more compute.

4. It does not transfer.

A model tuned by the cheap habit is tuned for one dataset. Change the sensor, the operating range, or the operating conditions, and the tuning is worthless.

The cheap habit produces models that work by accident. Accidental models work until they do not, and when they fail, they fail silently and catastrophically.

1.3 What the Notebook Showed

The projectile problem in the notebook is the simplest possible illustration of the cheap habit’s failure.

The true range is $R(\theta) = (v^2/g) \sin(2\theta)$: a sinusoid of period 180° in θ . This is the shape of the phenomenon. It is fixed by physics, not by the data. No amount of looking at the measurements in $[5^\circ, 35^\circ]$ can reveal that the range will fall back down at 80° ; only the physics tells us.

Every model in the demo assumed a shape—linear, polynomial, piecewise-constant, arbitrary smooth—that did not match the sine. Each could fit the training window. None could extrapolate. The 3rd degree polynomial’s apparent success was a coincidence of the specific test range we happened to choose.

The notebook then shows what happens when we start from the shape instead: choose a hypothesis class of sinusoids of period 180° , using the pair $(\sin 2\theta, \cos 2\theta)$ as the minimal basis. The same data, the same tools, and a completely different outcome.

The fix was not a bigger model, more data, or better regularization. The fix was to write down the shape first, then choose a hypothesis class that respects it.

This is the entire course in miniature. Every session that follows will elaborate it.

1.4 Our Approach

Machine learning is the process of learning useful structure from data. Representation is the central lens through which that structure becomes visible, usable, and connected to decision-making.

What This Course Is

- A hands-on tour of machine learning essentials, covering core techniques in regression, classification, and clustering.
- A practical guide to data preprocessing, moving from raw, messy inputs to meaningful, decision-ready structures.
- A training ground for engineers who need to diagnose and understand their systems, rather than just treating them as black boxes.
- A course where every algorithm earns its place by solving a representational problem the previous one could not.

What This Course Is Not

To keep our journey focused, we define clear boundaries for this course. This is **not** a survey of all available machine learning algorithms, a guide to winning Kaggle competitions, or a tutorial on calling library functions. While we leverage statistical concepts, this is not a statistics course.

Furthermore, we exclude specialized domains to keep our core focus on three main problems, i.e. regression, classification and clustering. We will not cover, except in passing:

- **Bayesian methods.** When uncertainty, belief updating, and small data are the primary problems, Bayesian methods are the right tool. That is a different course.
- **Reinforcement learning.** When an agent must take sequential actions to maximize a reward, RL is the standard. That is a different course.
- **Causal inference.** When the goal is to intervene in a system rather than merely observe its patterns, causal methods are required. That is a different course.

1.5 The Modern Engineer's Mandate

With LLMs and powerful hardware commoditizing the ability to write boilerplate code and brute-force hyperparameter searches, the value of the modern engineer has shifted.

At least today, LLMs cannot detect silent representational invalidity. They can write the code that produced the failed demo.

The purpose of this course is to train engineers who can build and implement algorithms from scratch, formulate the underlying problem correctly, choose the right representation, and recognize when a system is fundamentally broken long before it reaches production.

1.6 Prerequisites and Logistics

What You Need to Know

This course assumes the following, all at the working level:

- **Linear algebra.** Vectors, matrices, matrix multiplication, change of basis, eigenvalues and eigenvectors.
- **Calculus.** Partial derivatives, gradients, chain rule, basic optimization.
- **Probability and statistics.** Distributions, expectation, variance, conditional probability, basic estimation.
- **Python.** You should be able to read and write Python, use NumPy and Matplotlib, and run Jupyter notebooks.

We will not re-teach any of this. If a gap appears in your background, address it independently. The course moves quickly and assumes fluency.

The Delivery Stack

- **Online Course Book.** The central hub hosting all interactive materials, accessible directly via the [ML: A Geometric Viewpoint Book](#).
- **Lecture Notes.** Detailed conceptual and theoretical notes accompanying each session to solidify your understanding.
- **Jupyter Notebooks.** Hands-on code implementations, translating the underlying geometry into runnable, step-by-step algorithms.
- **Lecture Videos & Community.** Recorded sessions and asynchronous Q&A discussions shared on our dedicated **Telegram channel/group**.

Because there are no grades, the only thing this course can offer you is understanding. Everything is structured around that fact.

1.7 Exercises

1. **Hands-on.** Write a complete version of our latest pipeline on your own—without any help from an LLM. Then compare it, line by line, with the [Session 00 Notebook](#). What did you get right, and what did you silently get wrong?
2. **Compare features.** Fit the model using a single sine feature, $\sin 2\theta$. Then repeat with an added 3° phase shift, $\sin(2\theta + 3^\circ)$. What changes in the fit, and what does that tell you about how sensitive a hypothesis class is to feature design?
3. **Another shape.** Suppose the phenomenon were instead $f(x) = e^{-x^2}$. What minimal feature basis would you use to express this structure? Which standard hypothesis classes would capture it well, and which would fail?
4. **A spurious feature.** Construct a dataset where a model with excellent cross-validation accuracy fails catastrophically under a small, physically realistic shift in the input distribution. Describe the shift, and explain why no metric could have caught it.